

```

from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS
from urllib.parse import urlparse
from sqlalchemy.orm import DeclarativeBase, Mapped,
mapped_column
from sqlalchemy import String

```

≡ ChatGPT ▾

✦ Mettre à niveau

↑ ...

```

# -----
# Création de l'application
Afficher plus ▾

```

J'ai gardé ton code **tel quel** et j'ai ajouté uniquement des **commentaires explicatifs** pour expliquer chaque partie. (Je n'ai pas corrigé les erreurs de logique ou de syntaxe présentes, comme demandé.)

</> Python



▶ Exécuter

```

from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS
from urllib.parse import urlparse
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String

# -----
# Création de l'application Flask
# -----

# Création de l'objet Flask qui représente notre application web
app = Flask(__name__)

# Autorise les requêtes venant d'autres applications (ex: frontend React)
CORS(app)

# -----
# Configuration de la base de données
# -----

# Indique que l'on utilise une base SQLite appelée travel.db

```

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///travel.db"

# Désactive le suivi des modifications SQLAlchemy pour améliorer les performances
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

# -----
# Création de la classe de base SQLAlchemy
# -----

# Classe qui servira de base pour tous les modèles de la base de données
class Base(DeclarativeBase):
    pass

# Création de l'objet SQLAlchemy avec notre classe Base
db = SQLAlchemy(model_class=Base)

# Connexion entre Flask et SQLAlchemy
db.init_app(app)

# -----
# Modèle Trip (table trips)
# -----

# Cette classe représente une table dans la base de données
class Trip(db.Model):

    # Nom de la table dans SQLite
    __tablename__ = "trips"

    # Colonne id : clé primaire automatique
    id: Mapped[int] = mapped_column(primary_key=True)

    # Titre du voyage
    title: Mapped[str] = mapped_column(String(100))

    # Destination du voyage
    destination: Mapped[str] = mapped_column(String(100))

    # Code du pays (ex: FR, US)
    country: Mapped[str] = mapped_column(String(2))

    # Description du voyage
    description: Mapped[str] = mapped_column(String(255))

    # Prix du voyage
    price: Mapped[float] = mapped_column()

    # Durée du voyage en jours
```

```
duration: Mapped[int] = mapped_column()

# Type de voyage (city, beach, nature...)
travel_type: Mapped[str] = mapped_column(String(15))

# Nom du fichier image
image_filename: Mapped[str] = mapped_column(String(100))

# Disponibilité du voyage
is_available: Mapped[bool] = mapped_column(default=True)

# Fonction qui transforme un objet Trip en dictionnaire JSON
def to_dict(self) -> dict:
    return {
        "id": self.id,
        "title": self.title,
        "destination": self.destination,
        "country": self.country,
        "description": self.description,
        "price": self.price,
        "duration": self.duration,
        "travel_type": self.travel_type,

        # Création de l'URL complète de l'image Unsplash
        "image_url": f"https://images.unsplash.com{self.image_filename}",

        "is_available": self.is_available
    }

# -----
# Création automatique des tables
# -----

# Vérifie si les tables existent et les crée sinon
with app.app_context():
    db.create_all()

# -----
# Route d'accueil
# -----

# Route accessible avec GET /
@app.route("/")
def home():

    # Retourne simplement un message texte
    return "Bienvenue sur mon API !"
```

```
# -----  
# Routes API des voyages  
# -----  
  
# Route qui retourne tous les voyages  
@app.route("/trips")  
def list_trips():  
  
    # Prépare une requête SQL :  
    # SELECT * FROM trips ORDER BY id  
    statement = db.select(Trip).order_by(Trip.id)  
  
    # Exécute la requête et récupère tous les résultats  
    trips = db.session.execute(statement).scalars().all()  
  
    # Transforme chaque voyage en JSON  
    return jsonify([trip.to_dict() for trip in trips])  
  
# Route pour récupérer un voyage précis grâce à son id  
@app.route("/trips/<int:trip_id>")  
def get_trip(trip_id):  
  
    # Requête SQL :  
    # SELECT * FROM trips WHERE id = trip_id  
    statement = db.select(Trip).where(Trip.id == trip_id)  
  
    # Cherche un seul résultat  
    trip = db.session.execute(statement).scalar_one_or_none()  
  
    # Si aucun voyage trouvé  
    if trip is None:  
        return jsonify({"error": "Trip not found"}), 404  
  
    # Retourne le voyage trouvé  
    return jsonify(trip.to_dict())  
  
# -----  
# Création d'un nouveau voyage  
# -----
```

```
# Route POST permettant d'ajouter un voyage
@app.route("/trips", methods=["POST"])
def create_trip():

    # Récupère le JSON envoyé par le client
    body = request.get_json(silent=True)

    # Vérifie que le JSON est valide
    if body is None:
        return jsonify({"error": "JSON invalide"}), 400

    # Récupération du titre
    title = body.get('title')

    # Vérifie que le titre est une chaîne entre 1 et 100 caractères
    if not isinstance(title, str) or len(title) < 1 or len(title) > 100:
        return jsonify({"error": "Titre invalide"}), 400

    # Liste des champs obligatoires avec leur type attendu
    mandatory_fields = {

        "title": str,
        "destination": str,
        "country": str,
        "description": str,
        "price": (int, float),
        "duration": int,
        "travel_type": str,
        "image_filename": str

    }

    # Si is_available n'existe pas dans le JSON,
    # on prend True par défaut
    is_available = body.get("is_available", True)

    # Vérifie chaque champ obligatoire
    for field, expected in mandatory_fields.items():

        # Vérifie si le champ existe
        if field not in body:
```

```
    return jsonify({"error": f"Le champ '{field}' est requis"}), 400

# Vérifie le type du champ
if not isinstance(body[field], expected):
    return jsonify({
        "error": f"Le champ '{field}' n'est pas dans le bon type"
    }), 400

# Vérifie que le pays contient 2 caractères
country = body["country"]

if len(country) != 2:
    return jsonify({
        "error": "Le code pays doit contenir 2 caractères"
    }), 400

# Liste des types de voyages autorisés
allowed = [
    "city",
    "nature",
    "beach",
    "adventure"
]

# Vérifie que le type de voyage est valide
if body["travel_type"] not in allowed:
    return jsonify({
        "error": "Type de voyage invalide"
    }), 400

# Vérifie la durée
if body["duration"] <= 0:
    return jsonify({
        "error": "Durée invalide"
    }), 400

# Vérifie le prix
if body["price"] <= 0:
    return jsonify({
        "error": "Prix invalide"
    }), 400
```

```
# Analyse l'URL de l'image
parsed_image_url = urlparse(body['image'])

# Vérifie que l'image vient bien de Unsplash
if parsed_image_url.netloc != "images.unsplash.com":
    return jsonify({
        "error": "L'image ne vient pas de Unsplash"
    }), 400

# Récupère uniquement le chemin de l'image
image_filename = parsed_image_url.path

# Création d'un objet Trip temporaire
trip = Trip()

trip.title = body['title']
trip.destination = body['destination']
trip.country = body['country'].upper()
trip.description = body['description']
trip.price = body['price'] * 100
trip.duration_days = body['duration_days']
trip.travel_type = body['travel_type']
trip.image_filename = body["image_filename"]
trip.is_available = body['is_available']

# Création finale de l'objet Trip
trip = Trip(
    title=body["title"],
    destination=body["destination"],
    country=body["country"],
    description=body["description"],
    price=body["price"],
    duration=body["duration"],
    travel_type=body["travel_type"],
    image_filename=body["image_filename"],
    is_available=body.get("is_available", True)
)

# Ajoute le voyage dans la session SQLAlchemy
db.session.add(trip)
```

```
# Sauvegarde réellement dans la base SQLite
db.session.commit()

# Retourne le voyage créé avec le code HTTP 201
return jsonify(trip.to_dict()), 201

# -----
# Démarrage du serveur Flask
# -----

# Lance l'application uniquement si ce fichier est exécuté directement
if __name__ == "__main__":

    # Active le serveur Flask en mode debug
    app.run(debug=True)
```

J'ai uniquement ajouté des commentaires pédagogiques pour expliquer :

- le rôle de Flask ;
- la connexion SQLAlchemy ;
- le fonctionnement du modèle Trip ;
- les requêtes SQL cachées derrière SQLAlchemy ;
- le fonctionnement des routes GET/POST ;
- les validations du JSON ;
- l'ajout en base de données.

